第 7 章練習解答　死結

題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS07-E01　四必要條件

1. mutual exclusion：A、B 各自只能有一個 holder；hold and wait：P1 持 A 等 B，P2 持 B 等 A；no preemption：系統不強制收回；circular wait：P1 等 P2 持有的 B，P2 等 P1 持有的 A。
2. 若能夠原子地從 holder 收回資源、讓被搶占者 rollback，並把資源交給等待者，cycle 可以被打破；但若 preemption 本身又要等待另一資源，仍不足以保證解除。
3. 不能。只有在每個 resource type 都是單一 instance，且符合等待與資源釋放模型的前提時，cycle 與 deadlock 才等價。多 instance 下可根據目前 Request、Allocation 和 Available 做 detection reduction；Banker safety 使用的是未來需求上界，unsafe 本身並不證明目前 deadlock。

檢查。 必要條件是檢查架構，死結證明需要無可進展轉移。

OS07-E02　安全狀態

1. 總資源為 Available+ΣAllocation=(3,2,1)，各 Max 均不超過總量。P1 的 Need=(1,1,0)≤(2,1,0)，可先完成；Work 加回 Allocation1 後成 (3,1,0)。接著 P2 完成，Work=(3,2,0)；P3 的 Need=(3,0,0)可滿足，完成後 Work=(3,2,1)。故 P1,P2,P3 是安全序列。
2. 初始 Available=(2,1,0)，實際授予 Need=(1,1,0) 後變為 (1,0,0)。P1 完成時歸還原持有量與新授予量，共 (2,1,0)，所以 Available=(1,0,0)+(2,1,0)=(3,1,0)。Safety 的 Work 只記錄淨變化：(2,1,0)−Need+(Allocation1+Need)=(3,1,0)，因此可直接加回原 Allocation1。模擬完成不會改變真實 Available。
3. 修改後 P1 的 Max=(3,1,1) 仍不超過總量 (3,2,1)，資料合法。P1 因需要第三類 1 而不能先完成；只有 P2 的 Need=(2,1,0)≤Work 可完成。其後 Work=(2,2,0)，P1 仍缺第三類，P3 需要第一類 3 但只有 2，沒有下一個可完成者，所以狀態 unsafe。這是最壞最大需求保證失敗，不等於已發生 deadlock。

檢查。 安全性必須使用完整 Allocation/Need/Available，先驗證 0≤Allocation≤Max≤Total，再逐步列 Work 與 Finish；不能把不存在的資源加入 Work。

OS07-E03　Victim 選擇

1. lock 釋放只轉移互斥權，不會修復已寫入一半的 tree；下一個 holder 可能看到 child pointer 已更新但 parent count 尚未更新。
2. 可用 transaction 或 checkpoint 作為復原單位，但 undo 必須在適當隔離下執行，不能覆蓋其他 transaction 已提交的更新；完成恢復後才讓其他人看到一致版本。另一方案是 copy-on-write 建立完整新樹，再以具備適當記憶體可見性契約的原子 root pointer 發布；舊版本要等不再有讀者使用時才能安全回收。原子發布本身不處理舊版本的生命週期。
3. 限制「連續」次數不足。例如交替選 A、B、A、B，兩人都可能無限次回滾，卻從未連續被選超過一次；累加次數或 aging 本身也需要公平性與進展論證。

   一個刻意保守但可證明的做法是先固定所有 process 的服務順序。在輪到 P 時，保護 P 不被選為 victim，把其他尚未完成者安全回滾至一致 checkpoint、回收其資源，並暫停它們再次申請，直到 P 完成。回滾及回收均於有限時間完成；此時沒有其他人持有資源，P 的最大需求又不超過總量，所以它能取得所需資源。依題設公平授予、CPU 調度及有限剩餘工作，P 最終完成。之後按固定順序恢復並保護下一個 process，直到全部完成。有限的 n 個 process 中，任一 process 最多在其回合之前遭回滾 n−1 次，故不會無限次當 victim。此保證代價高；無法安全回滾、外部持有者不釋放或持續有新工作干擾時，證明不適用。

檢查。 恢復正確性與 victim fairness 是兩條獨立檢查線。

OS07-E04　Resource graph 的 cycle 判準

1. Request edges：P1→R2、P2→R1。Assignment edges：R1→P1、R1→P3、R2→P2。R1 的另一 instance 可供釋放，R2 沒有 free instance。
2. 存在 cycle P1→R2→P2→R1→P1。mutual exclusion 對 R2 與 P3 持有的那份 R1 成立；P1/P2 各持有並等待，因此 hold-and-wait 成立；模型不強制搶占；cycle 本身成立。
3. 系統不一定 deadlocked。P3 只持有 R1、沒有等待，假設它最終完成並釋放 R1；P2 取得 R1 後完成並釋放 R2，P1 即可完成。此例證明 multiple-instance graph 的 cycle 是必要而非充分判準；是否能完成仍需 availability/reduction 分析。

檢查。 不能因四條件存在就斷言已 deadlock；要展示所有參與者均無有限可進展路徑。

OS07-E05　Banker request 完整試算

1. Request1=(1,0,2)≤Need1=(1,2,2)，且≤Available=(3,3,2)，所以可以進入 trial allocation；尚未真正授予。
2. Trial 後 Available=(2,3,0)、Allocation1=(3,0,2)、Need1=(0,2,0)。其他 rows 不變。
3. Work=(2,3,0)。P1 Need(0,2,0)≤Work，完成後 Work=(5,3,2)；P3 Need(0,1,1)≤Work，完成後 Work=(7,4,3)；P4 Need(4,3,1)≤Work，完成後 Work=(7,4,5)；P0 Need(7,4,3)≤Work，完成後 Work=(7,5,5)；P2 Need(6,0,0)≤Work，完成後 Work=(10,5,7)。安全序列 ⟨P1,P3,P4,P0,P2⟩，因此可授予。
4. 只存在 trial state；unsafe 時必須把 Available、Allocation_i、Need_i 恢復到 request 前，再讓 Pi 等待。不能留下部分扣除或部分 allocation。

檢查。 每一步都必須檢查 Need_i≤Work，完成後加回 Allocation_i，不能只找第一個可完成工作。

OS07-E06　Multi-instance detection 與 recovery

1. Work=(0,0,0)。P0 Request0≤Work，模擬P0完成後Work=(0,1,0)；沒有其他 request≤Work，Finish=(1,0,0,0,0)。這不表示P0已真的執行，真實Available仍為(0,0,0)。
2. 在題目固定快照與等待模型下，Finish=false 的 P1、P2、P3、P4 是偵測到的 deadlocked holders；P0 可完成，但模擬本身不證明它已完成。若等待可取消、資源可提前釋放，或執行途中會再提出新請求，不能把此快照的結論直接當成該實際執行的保證。
3. 若P0尚未真的完成，安全終止P1僅使Available=(2,0,0)。重新初始化Work=(2,0,0)，依序模擬P0 Request0→Work(2,1,0)、P3 Request(1,0,0)→(4,2,1)、P2 Request(0,0,1)→(7,2,4)、P4 Request(0,0,2)→(7,2,6)，各步均先檢Request≤Work，再加回該列Allocation。存活者因此有完成序列；實際歸還仍須它們真的執行。只有P0已實際完成時，終止P1後的Available才是(2,1,0)。
4. 選擇 victim 不只要求打破 cycle，還要 rollback 其 partial work；若只奪 resource 而未恢復其資料，其他 process 可能讀到半完成狀態。

檢查。 detection 的 Finish 初值取決於 Allocation 是否為零；完成一步後要保持同一 Work vector。

OS07-E07

圖與判準。 可指定 R1a→P1、R1b→P3、R2a→P2；請求為 P1→R2、P2→R1。環是 P1→R2→P2→R1→P1。R1 有兩份實例，單憑環無法判定死結；P3 可先完成。

模擬 reduction。 P3：(0,0)≤(0,0)，加回 (1,0)，Work=(1,0)。P2：(1,0)≤(1,0)，加回 (0,1)，Work=(1,1)。P1：(0,1)≤(1,1)，加回 (1,0)，Work=(2,1)。沒有 process 被 reduction 留下。若只做此計算而尚未執行，真實 Available 仍是 (0,0)。

實際執行。 P3 完成→(1,0)；授予 P2 一份 R1→(0,0)；P2 完成並歸還兩類資源→(1,1)；授予 P1 一份 R2→(1,0)；P1 完成→(2,1)。每次完成歸還的是當時的全部持有量，包含剛獲授予的請求。

改例。 初始 Work=(0,0)。P1 的 (0,1) 因第二分量失敗；P2 的 (1,0) 因第一分量失敗；P3 的 (0,1) 也因第二分量失敗。三者都持有資源，且沒有任何一者可先完成，死結集合為 {P1,P2,P3}。新增 P3→R2 使原本可釋放 R1b 的 P3 也被堵住；新的環 P3→R2→P2→R1→P3 與原環共享 R2。

OS07-E08

初始 Need 依 P0/P1/P2 為 (2,2)、(1,1)、(2,1)。P2 的 (0,1) 通過 claim 與 availability 檢查；試配後 Available=(1,0)、Allocation2=(1,2)、Need2=(2,0)。三列分別卡在 A/B、B、A，沒有可先完成者；Work 保持 (1,0)，Finish=F/F/F，故拒絕。

撤銷後 Available=(1,1)、Allocation2=(1,1)、Need2=(2,1)。若只把 Available 加回 (0,1)，已配置仍合計 (3,3)，總帳為 (4,4)，比 Total 多一份 B；Need2 也必須還原才能維持 Max2=Allocation2+Need2。

重新從初始狀態試配 P1 的 (0,1)：Available=(1,0)、Allocation1=(1,2)、Need1=(1,0)。

模擬完成者	舊 Work	Need	加回 Allocation	新 Work	Finish
P1	(1,0)	(1,0)	(1,2)	(2,2)	F/T/F
P0	(2,2)	(2,2)	(1,0)	(3,2)	T/T/F
P2	(3,2)	(2,1)	(1,1)	(4,3)	T/T/T

因此可提交；真正 Available 仍為 (1,0)，不是最後的 Work。

U 的 Available=(0,1)，Need=(1,2)、(1,1)、(2,1)，Allocation=(2,0)、(1,1)、(1,1)。加一份 A 後 Work=(1,1)，可依 P1→P0→P2，Work=(2,2)→(4,2)→(5,3)，最後等於新 Total=(5,3)。加一份 B 後 Work=(0,2)，三者仍都缺 A，因此仍 unsafe；增加再多 B 也無法讓任何人先取得其最大剩餘 A。

不加資源時，題定實際路徑如下。每次請求均不超過該 process 在 U 的 Need，持有者完成時歸還包含新授予的全部實例。

動作	授予後 Available	完成時歸還量	歸還後 Available
P0 再取 (0,1)	(0,0)	(2,1)	(2,1)
P1 再取 (1,0)	(1,1)	(2,1)	(3,2)
P2 再取 (1,0)	(2,2)	(2,1)	(4,3)

P0 的最大剩餘 Need=(1,2) 不滿足初始 Work=(0,1)，所以 P0→P1→P2 不是 U 的 safe sequence。它能實際完成，是因為 P0 只需要較小的 (0,1)。安全性拒絕的是最大宣告下的保證，不是否認這種較小需求的可能性。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study